四车协同 · 卷尺桥 · 使用手册

> 面向**使用者**，不是开发者。讲三件事：**接线不能错的地方**、**怎么启动**、**怎么用命令和话题操作**。 配套： 架构方案.pdf （设计原理）、 four_car_ws/README.md （代码结构）

第一部分：开工前必读 —— 12 个坑

> 这一部分全是**会烧东西、会白忙半天、会查不出原因**的类型。按顺序看完再接线。

⚠️ 1. 共地（排第一，因为最常见）

所有设备的 GND 必须连在一起：

```
树莓派 GND ──┬── L298N GND
             ├── MOSFET 源极
             ├── 底盘串口模块 GND
             ├── 舵机控制器 GND
             └── 电池负极
```

不共地的话：电机乱转、串口收到乱码、GPIO 读数飘。**症状千奇百怪，但原因就这一个。**

⚠️ 2. 电磁铁的 MOSFET 电路

不能拿 GPIO 直接拉电磁铁 —— 线圈电流是安培级，树莓派一个引脚最多 16mA。直接接要么不吸，要么烧 GPIO。

电路图





四个必须做对的点

项	要求	做错的后果
MOSFET 选型	**逻辑电平** MOSFET：IRLZ44N、AO3400、IRF3708、IRL540N	用 IRF540 之类的**驱不动**——它的开启电压 2~4V，树莓派 3.3V 只到一半，MOS 会工作在线性区**发热烧毁**
续流二极管	线圈两端**反向**并一个（1N4007，肖特基 1N5819 更好）	断电瞬间线圈产生几百伏反压 → **打穿 MOSFET**，或者干扰到树莓派
栅极下拉电阻	栅极到 GND 并一个 10kΩ	树莓派未初始化时栅极悬空，MOS **误导通**，电磁铁一直吸着
栅极串阻	GPIO 到栅极串一个 100Ω	栅极电容瞬间充电电流过大，长期可能损伤 GPIO

再加一层保险

电磁铁吸合瞬间电流很大，会让电池电压**瞬间跌落**。如果树莓派和电磁铁共用一块电池，可能被拉重启。两个办法：

- **树莓派单独供电**（充电宝 / 另一块电池）—— 最省事
- 电磁铁电源端并一个 **1000μF 电解电容** —— 吸收瞬时跌落

⚠️ 3. L298N 的 ENA 跳线帽

```
L298N 上 ENA 那个位置有个跳线帽（黑色小方块）。

   插着 → ENA 常高电平，PWM 完全不起作用（电机永远全速）
   拔掉 → ENA 由你的 GPIO 控制，PWM 才有效

★ 用 PWM 调速就必须拔掉。
```

L298N 还有几个要注意：

- **板载 5V 不要给树莓派供电** —— 电机启停会把 5V 拉垮，树莓派会重启。树莓派自己供电
- L298N 自身有约 **1.4V 压降**（内部双极型晶体管），12V 供电时电机实际只拿到约 10.6V
- 电机线接 `OUT1` / `OUT2`，**不区分正负**，接反了只是转向相反（改 `invert` 参数或对调两根线都行）

⚠️ 4. 串口分配（车上有两个串口设备）

设备	设备节点	物理连接
底盘 4 个电机	`/dev/ttyUSB0`	USB 转 TTL/RS485 模块插 USB 口
俯仰舵机	`/dev/ttyAMA0`	GPIO14(TXD, 物理脚 8) / GPIO15(RXD, 物理脚 10)

绝对不能搞混。 代码里是**显式指定**的，不做自动探测 —— 因为自动探测可能探到另一个。

> 💡 树莓派 5 还有额外硬件串口 `/dev/ttyAMA10`，不想用 USB 的话底盘可以走它（需要配 dtoverlay）。

如果串口打不开：

```
sudo usermod -aG dialout $USER        # 然后退出重登！组权限只对新会话生效
ls -l /dev/ttyUSB0 /dev/ttyAMA0       # 看权限是不是 crw-rw---- root dialout
```

舵机串口还有个特殊坑（Ubuntu 上）：`serial-getty@ttyAMA0` 会占住串口，权限变成 `crw--w---- root tty`（tty 组只有写没有读）。修：

```
sudo systemctl disable --now serial-getty@ttyAMA0.service
echo 'KERNEL=="ttyAMA0", GROUP="dialout", MODE="0660"' | sudo tee /etc/udev/rules.d/99-serial.rules
sudo udevadm control --reload-rules && sudo udevadm trigger
```

⚠️ 5. 电机地址必须先设好

4 个电机挂**同一条串口总线**，靠**地址**区分。出厂默认全是 `1` —— 四个都是 1 就全乱了。

```
# 一个一个接上去，问出它现在是几号
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 readid

# 或者整板接好，扫一遍看在线的地址
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 scan

# 交互式设成 1/2/3/4（按提示一个一个来）
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 setup
```

对应关系（车轮位置 → 地址）：

```
      车头
   FL(1)   FR(2)
   RL(3)   RR(4)
      车尾
```

⚠️ 6. 减速比必须写进驱动器（不写车速差 18.8 倍）

这个是最容易忽略、后果最严重的。手册原文：

> 设置减速比后，所有速度和位置等都会根据减速比自动缩放，对应电机减速后的**输出轴**的速度和位置。

意思：把驱动器里的减速比设成 18.8 之后，你发和收的转速就都是**轮子**的量了。

```
# 读现在是几（出厂是 1.00，必须改成 18.80）
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 gear

# 设成 18.8
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 gear 18.8
```

节点启动时会自动检查并修复，日志里会打这一行，**每次启动都要看**：

```
[base_controller]: 减速比检查: {'FL': 'ok', 'FR': 'fixed 1.00->18.80', ...}
```

如果写的是 `MISMATCH` 那就没设上，车会快/慢 18.8 倍。

⚠️ 7. 只有地址 1 的电机会回应运动命令

手册规定：「所有的运动命令**只有地址 1 电机会回复**确认收到命令，避免总线冲突」。

这不是故障，是设计。 所以：

- 对 2/3/4 号发速度命令，**收不到** `02` **应答**是正常的
- 代码里已经处理了（只有地址 1 等应答，其余发完就走）
- 想确认 2/3/4 的状态，用**读取类**命令单独问（读取命令每个地址都会应答）

⚠️ 8. 树莓派 5 的 GPIO 库

树莓派 5 的 GPIO 挂在 RP1 芯片上，`RPi.GPIO` **完全不工作**。必须用 `gpiozero`（底层走 `lgpio`）：

```
sudo apt install -y python3-gpiozero python3-lgpio
```

没装的话节点会报错并提示这条命令。

⚠️ 9. 伸出时间会随电池电压漂

卷尺电机是**纯时间控制**（通电 N 秒）。电机的转速受电压影响：

```
满电 8.4V 时：转 N 秒伸出 L 毫米
电量掉一半：  转 N 秒只伸出约 0.8L 毫米
```

所以：

- 调试和实跑尽量保持电量差不多
- 时间参数**留点余量**，别卡在"刚好伸出来"的临界值
- 电池电压可以在启动时用 `read_battery_mv()` 看一眼

⚠️ 10. 跑之前先确认没有别的进程占串口

之前调试留下的节点可能还开着串口，会导致"设备被占用"或者两个进程抢数据。

```
pgrep -af 'base_node|arm_node|agent_node|mission'    # 看看有没有残留
pkill -f 'ros2 launch'; pkill -f base_node           # 清掉
```

⚠️ 11. 4 台车的 ROS_DOMAIN_ID 必须一致

```
export ROS_DOMAIN_ID=0        # 4 台车 + 主机，全部用同一个值
```

而且要在**同一个 WiFi**下。建议：

- 用 **5GHz**（2.4G 在 4 台车同时跑时丢包严重）
- 路由器上做 **DHCP 保留**（固定 IP），避免重启换 IP

⚠️ 12. 改完参数文件要重新 launch

`params.yaml` 是**启动时读一次**的，改完必须重启节点。想在线改单个参数：

```
ros2 param set /base_controller base.max_wheel_rad_s 10.0
```

第二部分：接线总表

树莓派引脚分配

功能	GPIO (BCM)	物理脚	接哪
俯仰舵机 TX	GPIO14	8	舵机控制器 RX
俯仰舵机 RX	GPIO15	10	舵机控制器 TX
卷尺电机 调速	GPIO12	32	L298N 的 **ENA**
卷尺电机 方向1	GPIO23	16	L298N 的 **IN1**
卷尺电机 方向2	GPIO24	18	L298N 的 **IN2**
电磁铁	GPIO17	11	MOSFET 栅极（串 100Ω）
GND	—	6/9/14…	全部设备的 GND
IMU	I2C: GPIO2/3	3/5	← 待定，还没接

> 引脚号在 `four_car_ws/src/four_car_bringup/config/params.yaml` 里改。

供电

```
电池 11.1V（或 12V）──┬── 底盘电机（经扩展板）
                      ├── L298N +12V（卷尺电机）
                      └── 电磁铁（经 MOSFET）

树莓派 ── 单独供电（充电宝 / 独立 DC-DC）
          ★ 不要从 L298N 板载 5V 取电
```

第三部分：首次调试流程

> **每一步都要停下来确认再往下。** 跳步的话出问题不知道是哪一层的事。

阶段 A：不通电检查（万用表量通断）

- [ ] 所有 GND 连通
- [ ] GPIO12 ↔ L298N ENA、GPIO23 ↔ IN1、GPIO24 ↔ IN2
- [ ] GPIO17 ↔ 100Ω ↔ MOSFET 栅极

- [ ] 续流二极管方向**正确**（负极接电源正、正极接漏极）
- [ ] **L298N 的 ENA 跳线帽已拔掉**

阶段 B：只给树莓派上电，不接电机电源

```
# 1. 串口在不在
ls /dev/ttyUSB* /dev/ttyAMA0

# 2. GPIO 库在不在
python3 -c "import gpiozero, lgpio; print('ok')"

# 3. 空载测 GPIO（不接电机电源，只看引脚电平）
python3 -c "
import sys; sys.path.insert(0,'/home/zai/four_car_ws/src/car_arm')
from car_arm.l298n import L298N
from car_arm.magnet import Magnet
m = L298N(12,23,24); g = Magnet(17)
m.forward(0.5); print('L298N 正转 ok')
g.on(); print('电磁铁吸合 ok'); g.off()
m.close(); g.close()
"
```

阶段 C：接电机电源，测电机（车架空，轮子离地）

```
# 1. 探地址
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 scan

# 2. 设地址（按提示一个个来）
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 setup

# 3. 设减速比
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 gear 18.8

# 4. 体检
python3 tools/zdt_id_tool.py --port /dev/ttyUSB0 health
```

阶段 D：单个轮子转一下，标定方向

```
ros2 run car_base base_node
# 另开一个终端：
ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}}"
```

看四个轮子是不是都朝**前**转。有反的，改 `params.yaml`：

```
base:
  invert: "FL,RL"      # 逗号分隔，填需要翻的轮位
```

阶段 E：标定麦轮符号

测试	发什么	期望	不对就改
前进	`linear.x: 0.2`	车头方向直走	`invert`
横移	`linear.y: 0.2`	往**左**平移	`flip_vy: true`
自转	`angular.z: 0.5`	**逆时针**转	`flip_wz: true`

阶段 F：测卷尺伸出时间（你说的"一点点加"）

```
ros2 run car_arm arm_node
# 另开终端：
ros2 topic pub --once /arm_cmd std_msgs/msg/String "{data: 'EXTEND'}"
```

从 3 秒开始，每次 +1 秒，直到卷尺正好伸到位：

```
# 在线改，不用重启节点
ros2 param set /arm_controller arm.extend_seconds 12.0
```

定下来后写进 `params.yaml`。

阶段 G：磁铁

```
ros2 topic pub --once /arm_cmd std_msgs/msg/String "{data: 'DOCK'}"     # 吸合
ros2 topic pub --once /arm_cmd std_msgs/msg/String "{data: 'RELEASE'}"  # 松开
```

如果**通电反而不吸、断电才吸** → 你的 MOSFET 是反相的，改 `params.yaml`：

```
arm:
  magnet_active_high: false
```

阶段 H：单台车完整跑

```
export CAR_ID=1
ros2 launch four_car_bringup car.launch.py
```

阶段 I：4 台车 + 主机

```
# 4 台车各跑（每台改 CAR_ID）
CAR_ID=1 ros2 launch four_car_bringup car.launch.py

# 主机（其中一台车，或上位机）
ros2 launch four_car_bringup mission.launch.py
```

第四部分：日常使用接口

4.1 启动命令

命令	用途
`ros2 launch four_car_bringup car.launch.py`	**一台车**的全部节点（底盘 + 上层 + 状态机）。靠 `CAR_ID` 环境变量区分车号
`ros2 launch four_car_bringup mission.launch.py`	**主机**（编排层）。4 台车之外单独跑一个
`ros2 launch four_car_bringup sim.launch.py`	**仿真**：4 台假车 + 主机，不用任何硬件，验证流程用

常用变体：

```
CAR_ID=2 ros2 launch four_car_bringup car.launch.py              # 2 号车
CAR_ID=1 ros2 launch four_car_bringup car.launch.py simulate:=true  # 不碰硬件
ros2 launch four_car_bringup mission.launch.py autostart:=false     # 不自动开跑
```

单独跑某个节点（调试用）：

```
ros2 run car_base  base_node          # 只跑底盘
ros2 run car_arm   arm_node           # 只跑上层
ros2 run car_agent agent_node         # 只跑状态机
ros2 run mission_controller mission_node   # 只跑主机
```

4.2 你手动操作的话题

控底盘 —— `/carN/cmd_vel`

类型 `geometry_msgs/Twist`。

```
# 前进 0.2 m/s
ros2 topic pub -r 10 /car1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}}"

# 倒车
ros2 topic pub -r 10 /car1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: -0.2}}"

# 往左横移
ros2 topic pub -r 10 /car1/cmd_vel geometry_msgs/msg/Twist "{linear: {y: 0.2}}"

# 原地逆时针转
ros2 topic pub -r 10 /car1/cmd_vel geometry_msgs/msg/Twist "{angular: {z: 0.5}}"

# 斜着走（前进+横移）
ros2 topic pub -r 10 /car1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.15, y: 0.15}}"

# 停车
ros2 topic pub --once /car1/cmd_vel geometry_msgs/msg/Twist "{}"
```

字段	单位	正方向
`linear.x`	m/s	车头方向
`linear.y`	m/s	左
`angular.z`	rad/s	逆时针

> **安全**：0.5 秒没收到新指令会**自动停车**。所以手动控制要用 `-r 10` 持续发，别用 `--once` 。

控上层 —— `/carN/arm_cmd`

类型 `std_msgs/String` 。

```
ros2 topic pub --once /car1/arm_cmd std_msgs/msg/String "{data: 'EXTEND'}"   # 伸出卷尺
ros2 topic pub --once /car1/arm_cmd std_msgs/msg/String "{data: 'RETRACT'}"  # 收回
ros2 topic pub --once /car1/arm_cmd std_msgs/msg/String "{data: 'DOCK'}"     # 电磁铁吸合
ros2 topic pub --once /car1/arm_cmd std_msgs/msg/String "{data: 'RELEASE'}"  # 电磁铁松开
ros2 topic pub --once /car1/arm_cmd std_msgs/msg/String "{data: 'PITCH'}"    # 重新摆俯仰角
```

触发整个任务 —— `/mission/state`

类型 `std_msgs/String` 。**广播给所有车**（不带命名空间）。

```
ros2 topic pub --once /mission/state std_msgs/msg/String "{data: 'GO'}"      # 开始
ros2 topic pub --once /mission/state std_msgs/msg/String "{data: 'EXTEND'}"  # 伸卷尺
ros2 topic pub --once /mission/state std_msgs/msg/String "{data: 'ABORT'}"   # 中止
```

> 主机 `autostart:=true` 时会自己发 `GO` ，不用手动。

4.3 你要看的话题

话题	类型	看什么
`/carN/odom`	`nav_msgs/Odometry`	车走到哪了（x, y, 朝向）
`/carN/status`	`std_msgs/String`	状态机当前状态（`MOVING` / `ALIGNED` / `DOCKED` …）
`/carN/arm_state`	`std_msgs/String`	上层状态（`EXTENDING` / `DOCKING` / `DOCKED` …）
`/carN/imu`	`sensor_msgs/Imu`	IMU 原始数据（如果接了）
`/rosout`	—	**全网所有节点的日志**（分布式调试神器）

```
ros2 topic echo /car1/odom
ros2 topic echo /car1/status
ros2 topic echo /rosout          # 一眼看 4 台车在干什么
ros2 topic hz /car1/odom         # 看频率正不正常
```

4.4 改参数

```
# 看有哪些参数
ros2 param list /base_controller
ros2 param list /arm_controller

# 看某个参数当前值
ros2 param get /arm_controller arm.extend_seconds

# 在线改（立即生效，重启后失效）
ros2 param set /arm_controller arm.extend_seconds 12.0
ros2 param set /base_controller base.max_wheel_rad_s 8.0

# 永久改 → 编辑 params.yaml 后重启
#   four_car_ws/src/four_car_bringup/config/params.yaml
```

最常改的几个：

参数	默认	说明
`arm.extend_seconds`	8.0	★ 卷尺伸出通电时间
`base.invert`	`""`	哪几个轮子反相，如 `"FL,RL"`
`base.flip_vy` / `base.flip_wz`	false	麦轮符号标定
`mission.drive_distance_m`	3.0	前车直行距离
`mission.target_yaw_deg`	90.0	转向角度
`base.max_wheel_rad_s`	16.0	限速（≈0.81 m/s 是电机上限）

4.5 工具脚本

```
cd ~/four_car_ws/src/car_base/tools

# 电机地址
python3 zdt_id_tool.py --port /dev/ttyUSB0 scan      # 扫描在线地址
python3 zdt_id_tool.py --port /dev/ttyUSB0 readid    # 广播读（只接一个电机时）
python3 zdt_id_tool.py --port /dev/ttyUSB0 setid 5 1 # 5号改成1号
python3 zdt_id_tool.py --port /dev/ttyUSB0 setup     # 交互式设成 1/2/3/4
python3 zdt_id_tool.py --port /dev/ttyUSB0 gear      # 读减速比
python3 zdt_id_tool.py --port /dev/ttyUSB0 gear 18.8 # 设减速比
python3 zdt_id_tool.py --port /dev/ttyUSB0 health    # 体检：连通性/温度/转速
```

4.6 停止

```
# 停所有 ROS 节点
pkill -f 'ros2 launch'; pkill -f base_node; pkill -f arm_node
pkill -f agent_node; pkill -f mission_node

# 急停（广播给所有车）
ros2 topic pub --once /mission/state std_msgs/msg/String "{data: 'ABORT'}"
```

第五部分：命令速查

```
# ============ 环境（每个新终端）============
source /opt/ros/jazzy/setup.bash
source ~/four_car_ws/install/setup.bash
export ROS_DOMAIN_ID=0

# ============ 看 ============
ros2 node list                     # 有哪些节点
ros2 node info /base_controller    # 这个节点订阅/发布什么
ros2 topic list -t                 # 话题 + 类型
ros2 topic echo /car1/odom         # 看内容
ros2 topic hz /car1/odom           # 看频率
ros2 topic info /car1/cmd_vel -v   # 谁在发、谁在收
ros2 param list /arm_controller    # 看参数
ros2 topic echo /rosout            # ★ 全网日志

# ============ 发 ============
ros2 topic pub -r 10 /car1/cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.2}}"
ros2 topic pub --once /car1/arm_cmd std_msgs/msg/String "{data: 'EXTEND'}"
ros2 topic pub --once /mission/state std_msgs/msg/String "{data: 'GO'}"

# ============ 改 ============
ros2 param set /arm_controller arm.extend_seconds 12.0

# ============ 跑 ============
ros2 launch four_car_bringup car.launch.py          # 单车
ros2 launch four_car_bringup mission.launch.py      # 主机
```

```
ros2 launch four_car_bringup sim.launch.py          # 仿真

# ============ 自测（不需要硬件）============
cd ~/four_car_ws && python3 selftest.py
```

附：出问题先查这四样

1. **共地了吗** —— 80% 的怪问题出在这
2. `ROS_DOMAIN_ID` **一样吗** —— 跨机通信不通基本都是这个
3. **串口对不对** —— 底盘 `/dev/ttyUSB0`、舵机 `/dev/ttyAMA0`，别搞混
4. **看** `/rosout` —— 4 台车的日志都在这一个话题上，比一个个 SSH 进去快得多

使用手册 v1.0 · 2026-09-19